Optimal Prompt Transformation in Model & Agent Communication Systems (OPTiMACS)

Anonymous ACL submission

Abstract

Current LLM frameworks predominantly rely on natural language for communication, an approach that is often ambiguous and unoptimized for specific task requirements. While alternative formats exist, the literature remains fragmented: (1) studies rely on narrow, ad-hoc selections of representations; (2) fail to generalize to new domains; (3) overlook multi-agent complexities like agentic scaffolding considerations; and (4) lack a theoretically grounded framework for efficient format discovery. We address these limitations through a unified optimization framework. Our contributions are threefold: (1) we establish the performance ceilings of static representations in both standard LLM prompting and multi-agent settings, highlighting the critical need for adaptive format discovery; (2) we propose novel algorithms and problem modeling approaches—formulating prompting optimization as a parallel-strategy tabular-contextual bandit and introducing the Expanding Markov Decision Process (EMDP) for multi-agent settings; and (3) we present OPTiMACS (Optimal Prompt Transformation in Model & Agent Communication Systems), a framework that leverages these formulations to dynamically learn optimal representations from interaction feedback. Evaluations across LLM-prompting problem solving and collaborative multi-agent tasks (multi-hop QA, mutual problem solving) show our methods identify task-aware formats that significantly outperform natural language, LLM-suggested formats, and static formats; achieving high efficiency without model fine-tuning.

1 Introduction

TODO : Add it Here

We want to capture:

- why
- what - format

Outlines

- generic
- problem and how helpful
- our contributions

2 Related Works

2.1 Format Bias and Structured Representations

Recent studies (Xu and Wang, 2024; Sahoo and Singh, 2024) demonstrate that LLMs exhibit significant "format bias," where specific structured representations often outperform natural language. However, relying on fixed formats across entire datasets can degrade performance (Beurer-Kellner and Vechev, 2024; Syst and et al., 2025). These wroks on finding optimal formats for tasks suffer from **three** critical **limitations**: (1) **Limited Coverage and Learning:** Existing methods test static, ad-hoc format collections without a mechanism to learn from success or failure; (2) **Lack of Domain Adaptation:** Static formats fail to address the complex, task-specific workloads of real-world private datasets; and (3) **Vulnerability to Drift:** Static selections are not invariant to data or model drift.

Approaches like Autoform (Chen et al., 2024) and similar incomplete-prompt techniques (Singh et al., 2024) attempt to address this by querying the LLM for optimal formats. However, they encounter a critical bottleneck: while LLMs can generate a broader coverage of candidate ideas than humans (Siu and et al., 2024), the quality of these generations is inconsistent, and an **LLM's *preference* often diverges from actual *performance***. Consequently, a robust, low-resource approach is required to adaptively explore and validate formats in drift-prone settings. OPTiMACS addresses this by efficiently learning optimal formats with minimal data points ($\approx$ 200–500) and epochs (3–5), without fine-tuning.

2.2 Orthogonal Works

Several research directions are orthogonal to our contribution and are complementary to OPTiMACS.

Custom Infrastructure: Works proposing efficient serialization standards (TOON (Lafalce and et al., 2025)) or infrastructure protocols (KQML (Finin et al., 1994), FIPA-ACL (FIPA, 1997), POML (Zhang et al., 2025), Verifiable Control (Wang and et al., 2024)) provide the *candidates* for communication but not the *intelligence* to select them. OPTiMACS functions as the policy layer to discover which of these candidates is optimal.

Prompt Optimization: Unlike methods such as APE (Zhou et al., 2023), OPRO (Yang et al., 2023), and RLPrompt (Deng et al., 2022) which optimize the *semantics* of instructions, we optimize the *syntax* (format) of the payload.

Reasoning Frameworks: Strategies like Chain-of-Thought (CoT) (Wei et al., 2022), Program-of-Thoughts (PoT) (Chen et al., 2022), X-of-Thought (Liu and et al., 2023), and Graph-of-Thoughts (GoT) (Besta and et al., 2024) enhance performance by structuring the *reasoning topology* or intermediate logic; our work is complementary, optimizing the message syntax used within these reasoning steps.

Agent Protocols and MARL: Modern standards like MCP (Anthropic, 2024), ACP (Consortium, 2024a), A2A (Group, 2024), and ANP (Consortium, 2024b) establish reliable connection handshakes and network topologies. In contrast, OPTiMACS optimizes the informational payload transmitted over these channels. Finally, unlike Multi-Agent Reinforcement Learning (MARL), which typically optimizes model parameters (Foerster et al., 2016) or communication topologies (Yang et al., 2021), we employ classical RL (Bandits/EMDP) to optimize communication at the inference level, ensuring efficiency without fine-tuning.

3 Problem Formulation

We present the preliminaries in section 3.1, with details in Appendix A. We then formalise the problem for single-step LLM calls in section 3.2, and for multi-agent system in section 3.3.

3.1 Preliminaries

We briefly formally introduce the key concepts used in our framework. Detailed mathematical formulations are provided in Appendix A.

Policy (π): A policy is a strategy, mapping states (contexts) to actions (decisions), denoted as $\pi(a|s)$, representing the probability of taking action a in state s.

A/B Testing: A randomized control experiment used to compare two versions of a variable (e.g., prompts) to determine which performs better on average across a dataset.

Contextual Bandits: A simplified reinforcement learning setting where an agent observes a context, selects an action, and receives an immediate reward, without state transitions. We use this validation for single-step prompting where the action (format) does not impact future states.

Markov Decision Processes (MDPs): A framework for sequential decision-making characterized by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}$ defines transition probabilities between states, $\mathcal{R}$ is the reward function, and γ is theoretically the discount factor. We employ this for multi-agent coordination where current decisions fundamentally alter the dialogue trajectory.

Q-values ($Q(s, a)$): The expected cumulative future reward an agent will receive by taking action a in state s and following a specific policy thereafter.

Off-Policy Learning: An off-policy algorithm learns the value of a target policy π (e.g., optimal formats) while following a different behavior policy μ for exploration. Both our algorithms are off-policy as they separate the learning of optimal strategies from the diverse exploration required to discover them.

3.2 Formulation for 1-step LLM Calls

Before moving ahead with muli-agentic setting, we first formalize the problem of LLM prompting and justify several key design choices as well as need for Reinforcement Learning (RL). While A/B testing can identify a globally superior static prompt, it fails to capture instance-specific preferences where different queries require different formats. A contextual-bandit formulation addresses this by learning a policy that maps specific contexts to optimal actions. Our formulation aligns with the standard Contextual Bandit framework—mapping contexts to actions to maximize reward—but is distinguished by two key differences: (1) **Tabular**

State Space: Unlike vector-based methods (e.g., LinUCB) that generalize across infinite states, we discretize the context into semantic task categories, defining valid clusters as proposed in Autoform (Chen et al., 2024) and program-based reasoning studies (Tian and et al., 2024); and (2) **Ensemble Exploration**: We utilize a union of diverse strategies rather than a single policy. The benefit of (1) is interpretable, data-efficient learning that avoids complex function approximation, while (2) ensures robust coverage of the search space.

Algorithm 1 formalizes this approach. It iteratively updates the tabular state space using feedback from the environment. This theoretical model ensures that we can bound the regret of our format selection.

Algorithm 1 OPTiMACS for 1-step LLM Call

1. **Input:** Training Dataset $\mathcal{D}$ (Training) / Query x (Inference)
2. **If Training:**
   (a) **Task Selection:** Cluster all $x_i \in \mathcal{D}$ into semantic tasks $\mathcal{T}$.
   (b) **Format Learning:** For each $x_i \in \mathcal{D}$:
   - $C \leftarrow \emptyset$ *// Candidate set*
   - **For** each strategy $k \in \{1 \dots K\}$:
     - $c_k \leftarrow \text{Strategy}_k(x_i, t_i, Q)$ *// e.g., Softmax, Meta-LLM*
     - $C \leftarrow C \cup \{c_k\}$
   - **For** $f \in C$:
     - $\hat{y} \leftarrow \text{LLM}(x_i, f)$, $r \leftarrow \text{Verify}(\hat{y}, y_i)$
     - $Q(t_i, f) \leftarrow Q(t_i, f) + r$ *// Update score*
3. **Else (Inference):**
   - $t \leftarrow \text{ClassifyTask}(x)$
   - **Return** $f^* = \text{argmax}_f\, Q(t, f)$

3.3 Multi-Agent System

Extending the 1-step formulation to multi-agent settings is non-trivial due to four critical limitations: (1) **State Transitions**: Unlike bandits where contexts are independent, agent actions (formats) fundamentally alter the dialogue trajectory and subsequent states; (2) **Delayed Rewards**: The success of an interaction is often only observable at the end of a multi-turn conversation, exacerbating the credit assignment problem; (3) **Lack of Strategic Lookahead**: Standard contextual bandits optimize myopically for the immediate step, failing to plan for long-term goal convergence essential in collaborative tasks; and (4) **Combinatorial Explosion**: The space of possible trajectories grows exponentially if we consider all strategies in parallel.

To address this, we model the system as a Markov Decision Process (MDP), where task clusters serve as states to capture the evolving context. This allows optimizing for long-term expected returns, and we mitigate the explosion of trajectories by using a weighted sampling of strategies based on scores or equal probability. However, standard MDPs fail here because they assume a fixed universe of states and actions. In our setting, this assumption breaks due to: (1) **Evolving State-Action Spaces**, as agents discover novel tasks and optimal formats online; (2) **Non-Stationarity**, arising from the continuous expansion of these spaces; and (3) **Latent State Uncertainty**, since task categories are inferred rather than observed, creating a partially observable challenge. To resolve these, we propose the Expanding Markov Decision Process (EMDP), utilizing specialized agents $\mathfrak{A}$ that alternate between exploration (expansion) and exploitation (convergence) phases. The behavior policy μ is a weighted mixture (w_Q, w_L, w_D) of greedy, learning, and diversity components.

Algorithm 2 outlines our proposed EMDP-based learning procedure. It employs an off-policy every-visit Monte Carlo control method with weighted importance sampling, adapted for an evolving state-action space. In this algorithm, we define the behavior policy $\mu(a|s)$ as a weighted sampling of three specific strategies: Inverse-Frequency, LLM-guided, and Sampling from Q-values. We utilize only these three as they were the top-performing strategies identified in the single-agent LLM-Prompting experiments.

4 The OPTiMACS Framework

The OPTiMACS framework implements the theoretical formulation described in the previous section. As illustrated in Figure 1, the system orchestrates interaction through a shared high-level architecture: a **Task Classifier** identifies the context, a **Policy Engine** selects the format, and a **Feedback Loop** drives learning. We now share the implementation specific details of the formulation above. However, the specific implementation of these components differs significantly between the single-agent and multi-agent settings. We detail each implementation below.

Algorithm 2 OPTIMACS for Multi-Agent Setting

1. **Initialize** for all $s \in \mathcal{T}, a \in \mathcal{F}(s)$:
   - $Q(s,a) \leftarrow$ arbitrary, $C(s,a) \leftarrow 0$
   - $\mu(a|s) \leftarrow w_Q \pi_Q(a|s) + w_L \pi_L(a|s) + w_D \pi_D(a|s)$ (behavior policy)
   - $\pi(s) \leftarrow$ deterministic policy greedy wrt Q
2. **Repeat** until convergence:
   (a) Sample msg from queries/problems.
   (b) **Generate Episode** using behavior policy $\mu(a|s)$:
      - **Repeat** until episode end:
        - $\mathcal{T}_t \leftarrow$ TaskCategorization(msg, context)
        - $\mathcal{F}_t \leftarrow \mu(T_t, context)$
        - prompt $\leftarrow$ ApplyStructure(msg, $\mathcal{F}_t$)
        - msg, $R_{t+1} \leftarrow \mathfrak{A}_t$(prompt)
        - **If** Expansion Phase: $\mathcal{T} \leftarrow \mathcal{T} \cup \{\mathcal{T}_t\}$, $\mathcal{F} \leftarrow \mathcal{F} \cup \{F_t\}$
        - $t \leftarrow t+1$
   (c) $G \leftarrow 0, W \leftarrow 1$
   (d) **For** $t = T-1$ down to 0:
      - $G \leftarrow \gamma G + R_{t+1}$
      - $C(\mathcal{T}_t, \mathcal{F}_t) \leftarrow C(\mathcal{T}_t, \mathcal{F}_t) + W$
      - $Q(\mathcal{T}_t, \mathcal{F}_t) \leftarrow Q(\mathcal{T}_t, \mathcal{F}_t) + \frac{W}{C(\mathcal{T}_t, \mathcal{F}_t)}[G - Q(\mathcal{T}_t, \mathcal{F}_t)]$
      - $\pi(\mathcal{F}_t|\mathcal{T}_t) \leftarrow 1$ if $\mathcal{F}_t = \arg\max_a Q(\mathcal{T}_t, a)$ else 0
      - $W \leftarrow W \cdot \frac{\pi(\mathcal{F}_t|\mathcal{T}_t)}{\mu(\mathcal{F}_t|\mathcal{T}_t)}$
      - **If** $W = 0$ **Exit For Loop**

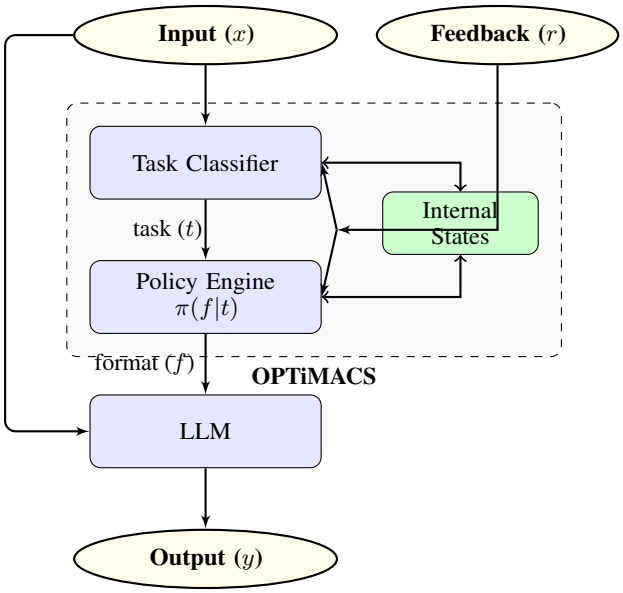

Figure 1: The OPTiMACS Framework. The system orchestrates interaction through three primary components: the Task Classifier, the Policy Engine, and a shared memory of Internal States.

4.1 OPTiMACS for Prompting (Single-Agent)

In the single-agent setting, the goal is to optimize the query format f for a given input x to maximize the correctness of the response y. We model this as a **Tabular Contextual Bandit**.

Architecture and State: The "state" s corresponds to the semantic task category of the input (e.g., *Code Generation*).

- **Classifier:** Uses a static, pre-defined taxonomy of tasks. It maps $x \rightarrow t$ using a 1-shot prompt.
- **Internal Memory:** Stores a fixed-size Q-table $Q(t, f)$ where rows are task categories and columns are the available format strategies.

Policy Engine (Ensemble Exploration): To efficiently explore the combinatorial space of prompts without infinite trials, we do not rely on a single policy. Instead, we implement the behavior policy μ as a weighted **Ensemble of 14 Strategies**, encompassing score-based exploitation, frequency-based exploration, and Meta-LLM guided inductive biases. Detailed definitions and formulations of these strategies are provided in Appendix D.

During inference, the system switches to a pure exploitation policy: $f^* = \arg\max_f Q(t, f)$.

4.2 OPTiMACS for Coordination (Multi-Agent)

In the multi-agent setting, the goal is to sequence agent roles and formats to solve complex, multi-turn problems. We model this as an **Expanding Markov Decision Process (EMDP)**.

Architecture and State (The EMDP): Unlike the single-agent case, the state space here is dynamic. The "state" s_t encompasses specifically verify the current sub-task definition, which evolves as the conversation progresses.

- **Dynamic Classifier (Expansion):** The classifier operates on the interaction history. Crucially, it includes an **Expansion Phase**: if the confidence in existing task categories is below a threshold δ, it proposes a new task node t_{new}.
- **Internal Memory (Internal States):** The memory structure σ grows dynamically. It stores the evolving taxonomy $\mathcal{T}$ and the Q-values for newly discovered states (initialized via semantic similarity to known states).

Policy Engine (Monte Carlo Control): The actions here represent agent personas and communication protocols.

- **Recursive Orchestrator:** The system recursively re-evaluates the state at each turn.
- **Credit Assignment:** Rewards are typically sparse (only received at the end of the trajectory). We therefore implement an **Every-Visit Monte Carlo** method. The system traces the full episode trajectory $\tau = (s_0, a_0, r_1, \ldots, s_T, r_T)$ and backpropagates the final reward to update $Q(s_t, a_t)$ for all intermediate steps.

This design enables OPTiMACS to adapt its complexity to the problem structure: utilizing data-efficient contextual bandits for 1-step prompting while leveraging the more expressive EMDP for multi-turn agent coordination.

5 Experiments

5.1 Datasets

We evaluate OPTiMACS across two distinct regimes: single-agent prompting and multi-agent coordination.

Single-Agent Prompting: We utilize three diverse reasoning benchmarks: (1) **AQuA-RAT** (Ling et al., 2017) for algebraic word problems; (2) **CoinFlip** (Wei et al., 2022) for symbolic manipulation and state tracking; and (3) **LogicGrid** (Srivastava et al., 2022) for constraint satisfaction and logical deduction. These datasets test the Policy Engine's ability to discover optimal static formats for distinct reasoning types.

Multi-Agent Coordination: We design two scenarios to test dynamic state management and collaboration:

- **Collaborative Problem Solving (GSM8k):** We instantiate a 3-agent team comprising a *Problem Solver*, a *Python Coder*, and a *Verifier* to solve grade-school math problems from the GSM8k dataset (Cobbe et al., 2021). This setup tests role-based sequencing and error correction.
- **Distributed Multi-Hop QA:** We employ **HotpotQA** (Yang et al., 2018) (multi-hop reasoning) and **NarrativeQA** (Kočiský et al., 2018) (long-context understanding from books/scripts). To enforce coordination, we partition the supporting documents (or book chapters) among agents, requiring them to exchange information via optimized communication formats to answer the query.

5.2 Setup

Our experiments aim to validate OPTiMACS's ability to discover optimal, instance-specific formats. We benchmark performance against three distinct baselines: (1) **Natural Language**; (2) **Autoform** (Chen et al., 2024) (LLM-proposed formats); and (3) **Fixed Formats** (Xu and Wang, 2024; Sahoo and Singh, 2024). We treat the latter as **A/B Testing**, where a single static format is enforced across all queries, and argue that this approach fails to capture the nuanced requirements of diverse instances.

Beyond performance comparisons, we conduct a comprehensive ablation study, focusing on: (1) the effectiveness of individual exploration strategies; (2) qualitative characteristics of the discovered tasks and formats; (3) information preservation, quantified using Style-Transfer metrics; and (4) the system's efficiency in terms of token usage and latency.

6 Results

TODO : Add it Here

(4) Efficiency of running OPTiMACS in terms of tokens and latency (5) RQ3: How is the efficiency of the system affected? 471 (iv) RQ4: What is the effectiveness of each com472 ponent? (v) RQ5: What is the distribution of the 473 dynamically evolving tasks and representations? 474 We detail the experiment setup below

7 Discussion

TODO : Add it Here

WHY FORMATS WORK:

- 1. Prompt Equaliastion - helpful bringing the reasoning paths for similar data and context (same reason why CoT and other Variants like PoT, XoT, GoT, ToT work). P9 works similarly
- 2. Ambiguity minimisation using structure (P5) where forcing some schemas for some tasks forces model to put attention to some information that it mith have left out due to schema/format requirements

TRANSFERRABILITY:

- Roles, Models have different formats (appendix) - transferrable across large models, GPT variants and 14B. But differences in SLMs like 4B, 7B ($LEFT$)

INSIGHTS:

- P1: Incomplete Prompt Technique to use LLMs to generate, similar to Autoform, NL not always the right format for structured-data, models have non-obvious preferences (we say that preference of model of choosing right might differ from what is actually right for the model). For tables, column-oriented worked, graph was good on adjancency dictionaries and edge lists were too. For SQL, Schemas, closed brakcet was good. Showed that pass@k and exact-match scores were affected. Models prefer compact structures as opposed to verbose inputs.

- P2: says reasoning drops in static format, but most likely this is issue with static prompts or same for all. We say different techniques help in different modes of reasoning. Since massively trained and instruction-tuned for NL instructions, the NL is good for in-general tasks, but for domain-specific tasks (our experiments), learning optimal formats are helpful. (Issue with fixed formats)

- Singh et al. (2024) showed that models often prefer compact structures—such as column-oriented formats for tables or adjacency lists for graphs—impacting pass@k and exact-match scores significantly.

- P3: 15 formats, and 253% variance in metrics , biases, we choose common from this list. (shows 'format bias') -> manually testing formats (bag). Similarly P4 uses 4 formats and says accuracy gains of 40% (json winner) + higher models more robust than smaller models .

8 Conclusion

TODO : Add it Here

9 Limitations

TODO : Add it Here

- As mentioned in P6 and P2, how good LLMs are at following the instruction of converting and if the instruction could be standardised

10 Future Works

TODO : Add it Here

- orthogonal direction

- Coding, depending on task, some languages might have better solution and efficient, like python for sorting list is 1 line vs assembly, but for modelling Memory, Assembly might be easier

- Mixture of formats, where we use not task, but learn information segregation, encoding that information optimally

- Unseen formats or consistency of generation using rules (example Json or Code, for JSON it should have nexting of atmost 2 level and for code give python code.)

- Designing novel formats suited to a purpose, like TOON for Token Efficiency or POML for context-passing

References

Anthropic. 2024. *Model Context Protocol (MCP)*. Technical Specification.

Maciej Besta and et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.09687*.

Luca Beurer-Kellner and Martin Vechev. 2024. Let me speak freely? a study on the impact of format restrictions on performance of large language models. *arXiv preprint arXiv:2408.02442*.

Weize Chen, Chenfei Yuan, Jiarui Yuan, Yusheng Su, Chen Qian, Cheng Yang, Ruobing Xie, Zhiyuan Liu, and Maosong Sun. 2024. Beyond natural language: LLMs leveraging alternative formats for enhanced reasoning and communication. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2022. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

ACP Consortium. 2024a. Agent communication protocol. *Technical Specification*.

ANP Consortium. 2024b. Agent network protocol. *Technical Specification*.

Mingkai Deng, Jianyu Wang, Cheng-Ping Hsieh, Yihan Wang, Han Guo, Tianmin Shu, Meng Song, Eric P Xing, and Zhiting Hu. 2022. RLPrompt: Optimizing discrete text prompts with reinforcement learning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.

Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. 1994. KQML as an agent communication language. In *Proceedings of the Third International Conference on Information and Knowledge Management*.

FIPA. 1997. *FIPA ACL Message Structure Specification*. Foundation for Intelligent Physical Agents.

Jakob Foerster, Ioannis Assael, Nando de Freitas, and Shimon Whiteson. 2016. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*.

A2A Working Group. 2024. Agent-to-agent protocol. *Technical Specification*.

Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. 2018. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328.

A. Lafalce and et al. 2025. TOON: Token-oriented object notation. https://github.com/toon-format/toon.

Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. Program induction by rationale generation: Learning to solve and explain algebraic word problems. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 158–167.

Jiayi Liu and et al. 2023. X-of-thought: Enabling chain-of-thought reasoning in large language models. *arXiv preprint arXiv:2311.04254*.

Pranav Sahoo and Arpit Singh. 2024. Does prompt formatting have any impact on LLM performance? *arXiv preprint arXiv:2411.10541*.

Usneek Singh, A. Singha, A. Awasthi, S. Gulwani, A. Kanade, V. Le, M. Singh, and G. Verbruggen. 2024. Asking language models how to represent data for fine-tuning. *arXiv preprint arXiv:2402.10842*.

Chenglei Siu and et al. 2024. Can LLMs generate novel research ideas? a large-scale human study with 100+ NLP researchers. *arXiv preprint arXiv:2409.04109*. Shows LLMs generate more novel ideas (coverage) but often lack feasibility compared to experts.

Aarohi Srivastava and 1 others. 2022. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*.

J. Syst and et al. 2025. Prompt engineering for structured data: A comparative evaluation. *arXiv preprint arXiv:2501.00000*. Preprint.

Y. Tian and et al. 2024. Can LLMs reason in the wild with programs? *arXiv preprint arXiv:2406.13764*.

Zhaoyang Wang and et al. 2024. Verifiable format control for large language model generations. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.

Christopher J.C.H. Watkins and Peter Dayan. 1992. Q-learning. *Machine Learning*, 8(3-4):279–292.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35.

Cuiyun Xu and Yujia Wang. 2024. LLMs are biased towards output formats! systematically evaluating and mitigating output format bias of LLMs. *arXiv preprint arXiv:2408.08656*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. 2023. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*.

Yiqun Yang, Huazhe Hu, Wengang Weng, and Heling Li. 2021. Learning correlated communication topology in multi-agent reinforcement learning. In *Proceedings of the 20th International Conference on Autonomous Agents and Multiagent Systems*, pages 456–464. IFAAMAS.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.

Y. Zhang, T. Xia, and et al. 2025. Prompt orchestration markup language (POML). *arXiv preprint arXiv:2508.13948*.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *Proceedings of the 11th International Conference on Learning Representations*.

A Standard Frameworks

A.1 Contextual Bandits

Formally, a Contextual Bandit problem is defined as a tuple $\langle \mathcal{X}, \mathcal{A}, \mathcal{R} \rangle$, where:

- $\mathcal{X}$ is the context space.
- $\mathcal{A}$ is the action space.
- $\mathcal{R} : \mathcal{X} \times \mathcal{A} \to [0, 1]$ is the reward function.

At each time step t, the agent observes a context $x_t \in \mathcal{X}$, selects an action $a_t \in \mathcal{A}$, and receives a reward $r_t \sim \mathcal{R}(x_t, a_t)$. The goal is to maximize the cumulative reward $\sum_{t=1}^{T} r_t$.

A.2 Markov Decision Processes (MDPs)

An MDP is defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:

- $\mathcal{S}$ is the state space.
- $\mathcal{A}$ is the action space.
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state transition probability function.
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the reward function.
- $\gamma \in [0, 1)$ is the discount factor.

A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to probability distributions over actions. The value function $V^{\pi}(s)$ represents the expected returns starting from state s following policy π.

B OPTiMACS Formulation

B.1 Multi-Agent Setting

We generalize the standard frameworks to a multi-agent setting where N specialized LLM agents $\{\mathfrak{A}_i\}_{i=1}^{N}$ collaborate.

- **State Space** $\mathcal{S}$: Defined by the semantic task clusters of user queries.
- **Action Space** $\mathcal{A}$: Represents message structures/formats $f \in \mathcal{F}$, such as `logical_reasoning` or `information_request`, used for inter-agent communication.
- **Transitions** $\mathcal{P}$: The transition $P(s'|s, a)$ models how the dialogue context (task cluster) evolves based on the chosen communication format.
- **Reward** $\mathcal{R}$: A sparse reward signal $+1$ is given only upon successful task completion, with 0 otherwise, emphasizing the need for effective credit assignment.

B.2 Expanding Markov-Decision Process (EMDP)

In our problem setup, we allow the task categories (states) and structure formats (actions) to evolve as learning progresses. This dynamic nature necessitates an extension of the standard MDP.

Expansion Mechanism:

- **State Expansion:** $\mathcal{T}_{new} = \mathcal{T}_{old} \cup \{t_{new}\}$ upon encountering a new task cluster t_{new}.
- **Action Expansion:** $\mathcal{F}_{new} = \mathcal{F}_{old} \cup \{f_{new}\}$ upon discovering a new effective format f_{new}.

We adopt an episodic approach where we allow expansion for a percentage of data points (exploration), followed by a fixed phase (exploitation) where the MDP definition is static, allowing standard Q-learning convergence guarantees to apply within the epoch. (TODO: Add ablation figure)

Algorithm Details: Algorithm 2 formally describes the learning process. The core components are:

1. **Initialization**: Q-values $Q(s, a)$ are initialized arbitrarily, and the cumulative weight $C(s, a)$ is set to 0. A behavior policy μ is defined as a mixture of the greedy policy π_Q, a learning/expansion policy π_L, and a diversity policy π_D.
2. **Episode Generation**: For each query, the agent generates a trajectory using μ. The task t and format f are selected, and the message is processed.
3. **Expansion**: During the expansion phase, new tasks and formats discovered by μ are added to the global sets $\mathcal{T}$ and $\mathcal{F}$.
4. **Update**: We use off-policy weighted importance sampling. The return G is computed, and Q-values are updated iteratively. The weight W adjusts for the discrepancy between the target policy π and behavior policy μ.

C Theoretical Analysis

We analyze the convergence properties of our approach.

C.1 Convergence of EMDP Algorithm

To prove the convergence of our EMDP-based algorithm, we leverage the property that during the exploitation phase (fixed states and actions), the problem reduces to a standard finite MDP.

Proposition 1 (Convergence of the proposed EMDP based algorithm)**.** *Consider the* $EMDP = (S_e, A_e, P_e, R, \gamma)$*, with value function* V*. For any* $\epsilon > 0$*, there exists a stationary policy* π^* *such that* $V^{\pi^*}(s) \geq V^*(s) - \epsilon$ *for all* $s \in S_e$*, where* V^* *is the optimal value function.*

Proof Sketch: During the fixed phase of the epoch, the state space S_e and action space A_e are constant. Standard Q-learning (Watkins and Dayan, 1992) is known to converge to the optimal action-value function Q^* with probability 1 under standard conditions (bounded rewards, appropriate learning rates). Our algorithm effectively performs standard Q-learning within these stable intervals. TODO: Add detailed formal proof.

D Single-Agent Strategies

D.1 Format Selection Strategies

We define a comprehensive set of $K = 14$ distinct format selection strategies $\mathcal{S} = \{s_1, s_2, \ldots, s_{14}\}$, each providing a different approach for choosing which prompt formats to evaluate for a given query q with task t. Let $\mathcal{F}$ denote the set of available prompt formats, $S_t[k]$ the score of format k for task t, and $U_t[k]$ the usage count of format k for task t.

Score-Based Selection Strategies

1. **Sampled from Score-Table** (s_1): Samples formats proportionally to their task-specific scores:

$$P(f_k \mid t) = \frac{S_t[k]}{\sum_{j \in \mathcal{F}} S_t[j]}, \quad f \sim P(\cdot \mid t) \tag{1}$$

2. **Inverse Frequency - Task** (s_2): Prioritizes less-used formats for the current task:

$$P(f_k \mid t) = \frac{1/U_t[k]}{\sum_{j \in \mathcal{F}} 1/U_t[j]}, \quad f \sim P(\cdot \mid t) \tag{2}$$

3. **Sampled Mixed - Task-Frequency Score** (s_3): Averages normalized scores across all tasks:

$$\bar{S}[k] = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \frac{S_t[k]}{\max_j S_t[j]} \tag{3}$$

$$P(f_k) = \frac{\bar{S}[k]}{\sum_j \bar{S}[j]} \tag{4}$$

4. **Sampled Mixed - Format Score** (s_4): Averages raw scores across all tasks without normalization:

$$\bar{S}[k] = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} S_t[k] \tag{5}$$

$$P(f_k) = \frac{\bar{S}[k]}{\sum_j \bar{S}[j]} \tag{6}$$

Frequency-Based Selection Strategies

5. **Inverse Frequency - Global** (s_5): Prioritizes globally under-explored formats:

$$U_{\text{global}}[k] = \sum_{t \in \mathcal{T}} U_t[k], \tag{7}$$

$$P(f_k) = \frac{1/U_{\text{global}}[k]}{\sum_j 1/U_{\text{global}}[j]} \tag{8}$$

6. **Frequency - Global** (s_6): Prioritizes frequently used formats across all tasks:

$$P(f_k) = \frac{U_{\text{global}}[k]}{\sum_j U_{\text{global}}[j]} \tag{9}$$

7. **Random** (s_7): Uniformly samples from all available formats:

$$P(f_k) = \frac{1}{|\mathcal{F}|}, \quad \forall k \in \mathcal{F} \tag{10}$$

LLM-Guided Selection Strategies For LLM-guided strategies, we use a meta-LLM $\mathcal{M}$ to suggest format selections. Let $R_t[k] = (S_t[k], U_t[k])$ denote the performance record for format k on task t.

8. **LLM-Task** (s_8): LLM suggests formats based on task description:

$$\mathcal{F}_{\text{selected}} = \mathcal{M}(\text{task\_desc}(t)) \tag{11}$$

9. **LLM-Query** (s_9): LLM suggests formats based on the specific query:

$$\mathcal{F}_{\text{selected}} = \mathcal{M}(q) \tag{12}$$

10. **LLM-Task with Performance Feedback (Task)** (s_{10}): LLM considers task and task-specific format performance:

$$\mathcal{F}_{\text{selected}} = \mathcal{M}(\text{task\_desc}(t), \{R_t[k]\}_{k \in \mathcal{F}}) \tag{13}$$

11. **LLM-Query with Performance Feedback (Task)** (s_{11}): LLM considers query and task-specific format performance:

$$\mathcal{F}_{\text{selected}} = \mathcal{M}(q, \{R_t[k]\}_{k \in \mathcal{F}}) \tag{14}$$

12. **LLM-Task with Performance Feedback (Global)** (s_{12}): LLM considers task and global format performance:

$$R_{\text{global}}[k] = \sum_{t' \in \mathcal{T}} R_{t'}[k] \tag{15}$$

$$\mathcal{F}_{\text{selected}} = \mathcal{M}(\text{task\_desc}(t), \{R_{\text{global}}[k]\}_{k \in \mathcal{F}}) \tag{16}$$

13. **LLM-Query with Performance Feedback (Global)** (s_{13}): LLM considers query and global format performance:

$$\mathcal{F}_{\text{selected}} = \mathcal{M}(q, \{R_{\text{global}}[k]\}_{k \in \mathcal{F}}) \tag{17}$$

14. **Natural Language** (s_{14}): Direct natural language description of the task to guide format selection:

$$\mathcal{F}_{\text{selected}} = \mathcal{M}(\text{nl\_desc}(t, q)) \tag{18}$$

Each strategy s_i represents a different hypothesis about which format selection approach will be most effective, ranging from purely statistical methods to LLM-guided intelligent selection incorporating historical performance data.